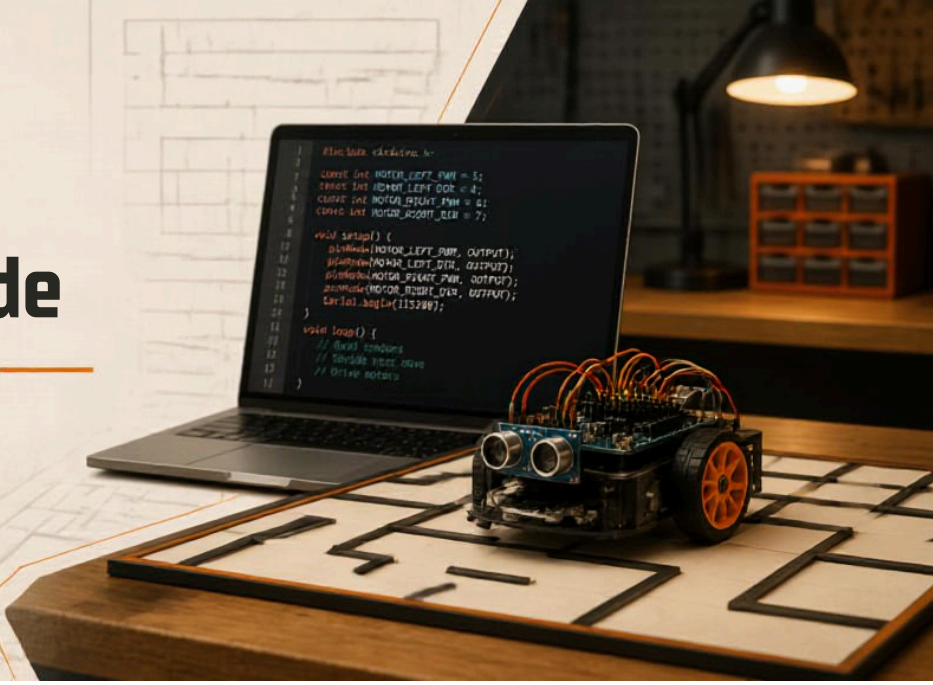
# DuelMaze

## Programming Guide

EXPANDED EDITION

C++ and Arduino foundations for building, testing, and debugging DuelMaze



# Programming Guide:

## C++ and Arduino foundations for building, testing, and debugging DuelMaze

**Makerspace CFAL**

*For school and beginner makerspace members.*

### Welcome

Coding is learned by making small changes, observing what happens, and debugging patiently. This guide turns each new idea into a practical step toward building DuelMaze with an Arduino, two servo motors, potentiometers or joystick axes, and a 16x2 I2C LCD.

## How to use this guide

### Try this learning loop

1. Read one section.
2. Type the example yourself instead of copy-pasting it.
3. Change one thing and observe what happens.
4. When stuck, check this guide, then the Serial Monitor, then ask a teammate, then ask a UC.

## Objectives

By the end, you should know enough C++ and Arduino to build DuelMaze using variables, logic, loops, arrays, functions, timing, inputs, servos, and the I2C LCD.

## Contents overview


| Part | Focus |
|---|---|
| 1 | How an Arduino program works |
| 2 | Variables and data types |
| 3 | Operators |
| 4 | Making decisions |
| 5 | Loops |
| 6 | Arrays |
| 7 | Functions |
| 8 | Serial Monitor and debugging |
| 9 | Timing: delay() versus millis() |
| 10 | Reading input |
| 11 | Servo motors |
| 12 | The 16x2 I2C LCD |
| 13 | Mini DuelMaze starter |
| 14 | Common errors and fixes |
| 15 | Quick reference cheat sheet |
| — | Practice Challenges |

# 1. How an Arduino Program Works

An Arduino program is called a **sketch**. You write a sketch in the Arduino IDE, click **Verify** to compile it and check for mistakes, and then click **Upload** to send it to the board.

Every sketch has two special functions. **setup()** runs once when the board powers on or resets. It is a good place to attach servos, start the LCD, and set pin modes. **loop()** runs again and again forever. It is where you read inputs, move servos, and update the screen.

```
void setup() {
  // runs once
}
void loop() {
  // runs forever
}
```

## Line-by-line explanation

- **void** means the function returns no value.
- **setup** is the special one-time function.
- The opening and closing braces **{ }** define the function body, or the group of instructions belonging to it.
- The comment documents the purpose of the function. A comment is for humans and is ignored when the sketch runs.
- **loop** is the special repeating function. After its closing brace, Arduino starts it again.

## Golden rules of C++ syntax

| Rule | Meaning |
| --- | --- |
| Every statement ends with ; | The semicolon marks the end of an instruction. |
| { } | Curly braces group code into blocks. |

| | |
|---|---|
| Case matters | C++ is case sensitive, so myServo and myservo are different names. |
| // and /* ... */ | Use // for one-line comments and /* ... */ for comments across many lines. |

**Try it**

Add a Serial message in setup and observe that it appears once. Then add a Serial message in loop and observe that it repeats. The difference shows the jobs of the two functions.

# 2. Variables and Data Types

A variable is like a labelled box that stores a value. C++ requires you to declare the type of box before using it, so the computer knows what kind of value belongs inside.

```
int a, b;          // declare now, assign later
a = 10; b = 16;
String myName = "Rajesh";   // declare and assign together
float x = 3.0 / 5.0;        // x = 0.6
```

## Line-by-line explanation

- **int a, b;** declares two whole-number variables named a and b. They exist, but no useful values have been assigned yet.
- **a = 10; b = 16;** assigns 10 to a and 16 to b. Assignment stores a value in a variable; the single equals sign means “put this value here.”
- **String myName = "Rajesh";** declares a text variable and assigns its starting text at the same time. On Arduino, String has a capital S.
- **float x = 3.0 / 5.0;** declares a decimal variable, performs decimal division, and stores 0.6 in x.

Declaring means introducing a variable and its type. Assigning means putting a value into that variable. You can do them separately or together.

| Type | What it stores | DuelMaze example |
|---|---|---|
| int | Whole numbers, -32768 to 32767 on Arduino Uno | score, angle |
| long | Big whole numbers | time in milliseconds |
| unsigned long | Whole numbers used with millis() | timestamps |
| float / double | Decimals; on Uno both are the same | 0.6 |
| bool | true or false | gameOver |

| | | |
|---|---|---|
| char | A single character | 'A' |
| String | Text | "DuelMaze" |
| byte | Whole numbers from 0 to 255 | small counter |

**Watch out**

**Integer division** can surprise you. 3/5 gives 0 because both numbers are integers, so the decimal part is thrown away. Write 3.0/5.0 to get 0.6.

| Expression | Result |
|---|---|
| 3/5 | 0 |
| 3.0/5.0 | 0.6 |

```
const int SERVO_PIN = 9;
```

Values that never change should be constants. **const** makes the intention clear and helps keep important settings safer from accidental changes. Here, SERVO_PIN always means pin 9.

## Scope: where a variable exists

A variable declared outside all functions is **global** and can be used throughout the sketch. A variable declared inside a function, loop, or other pair of braces is **local** and only exists inside those braces. Keeping short-lived values local helps prevent accidental changes elsewhere.

**Try it**

Create a global score and a local temporary variable inside loop(). Print both from inside loop(). Then try printing the local variable from setup() and observe the scope error. Move the declaration to a shared location and test again.

# 3. Operators

Operators let a sketch calculate, compare, and combine values. They are the small symbols that help DuelMaze decide what to do next.

## Arithmetic operators

Use **+** to add, **-** to subtract, ***** to multiply, **/** to divide, and **%** to find a remainder. For example, 7 % 3 is 1 because 3 fits into 7 twice, with 1 left over. A remainder is useful for checking whether a number is even or odd: an even number has number % 2 equal to 0.

Shortcuts make common updates compact: **x += 5** adds 5 to x, **x -= 1** subtracts 1, **x++** increases x by 1, and **x--** decreases x by 1.

## Comparison operators

| Operator | Meaning | Example |
|---|---|---|
| == | equal to | score == 10 |
| != | not equal to | buttonState != HIGH |
| < | less than | angle < 90 |
| > | greater than | score > 0 |
| <= | less than or equal to | angle <= 180 |
| >= | greater than or equal to | score >= 10 |

## Logic operators

- **&& AND** is true only when both conditions are true.
- **|| OR** is true when at least one condition is true.
- **! NOT** reverses a value: true becomes false and false becomes true.

**Watch out**

**=** assigns a value, while **==** compares values. **if (score = 10)** changes score

to 10 and then tests the resulting value, which is a bug in this situation. The correct comparison is **if (score == 10)**.

## Small game examples

```
if (score >= 10 && buttonState == LOW) {
  Serial.println("Bonus!");
}
```

- **score >= 10** checks whether the player has at least 10 points.
- **buttonState == LOW** checks whether the button is pressed when INPUT_PULLUP is being used.
- **&&** requires both tests to be true before the message is printed.
- The braces contain the action that happens when the combined condition is true.

# 4. Making Decisions

Use **if**, **else if**, and **else** when the sketch must choose between different actions. Conditions are checked from top to bottom, and only the first true block runs.

```
if (number > 0) {
  Serial.println("Positive");
} else if (number < 0) {
  Serial.println("Negative");
} else {
  Serial.println("Zero");
}
```

## Line-by-line explanation

- **if (number > 0)** asks whether number is greater than zero.
- **Serial.println("Positive");** prints Positive when that first condition is true.
- **else if (number < 0)** is checked only when the first condition was false, and asks whether number is less than zero.
- **Serial.println("Negative");** prints Negative for a negative number.
- **else** catches every remaining case. If number is neither above nor below zero, it must be zero.
- Each pair of braces groups the instructions for one branch.

## Switching between fixed options

When one variable can have several fixed choices, **switch** can be easier to read than a long chain of comparisons.

```
switch (gameState) {
  case 0: showMenu(); break;
  case 1: playGame(); break;
  case 2: showWinner(); break;
}
```

- **switch (gameState)** tells Arduino to inspect the current value of gameState.
- **case 0:** matches gameState equal to 0 and calls showMenu().
- **case 1:** matches 1 and calls playGame().

- **case 2:** matches 2 and calls showWinner().
- **break** stops the current case so execution does not continue into the next case.

**Tip**

Forgetting break makes code “fall through” into the next case. That can call several game actions when you intended to call only one.

**Try it**

Make a variable called gameState. Give it the values 0, 1, and 2 one at a time, and use switch to print menu, play, and winner messages.

# 5. Loops

Loops repeat work without making you write the same lines again. This is useful for counting, sweeping a servo, checking inputs, and repeating game actions.

## For loops

```
for (int i = 1; i <= 5; i++) {
  Serial.println(i);
}
```

- **int i = 1** is the start: create i and begin at 1.
- **i <= 5** is the condition: continue while i is less than or equal to 5.
- **i++** is the update: increase i by 1 after each round.
- **Serial.println(i);** prints the current value during each round.

The output is 1, 2, 3, 4, 5. When i becomes 6, the condition is false and the loop stops.

## While loops

```
int i = 1;
while (i <= 5) {
  Serial.println(i);
  i++;   // change i or the loop never ends
}
```

- **int i = 1;** creates the counter and gives it a starting value.
- **while (i <= 5)** checks the condition before each round.
- **Serial.println(i);** prints the current counter.
- **i++;** changes i so the condition eventually becomes false. Without it, the loop never ends.
- The closing brace marks the end of the repeated block.

## Do-while, break, and continue

A do-while loop runs its body at least once and checks the condition afterward.

```
int attempts = 0;
do {
  Serial.println("Try");
  attempts++;
} while (attempts < 3);
```

The variable starts at zero, the body prints Try and increases attempts, and only then does while check whether another round is needed. **break** exits a loop early; **continue** skips the rest of the current round and moves to the next one.

```
for (int i = 1; i <= 10; i++) {
  if (i % 2 != 0) continue;
  Serial.println(i);
}
```

The remainder test identifies odd numbers, continue skips them, and only even numbers reach Serial.println().

**Tip**

loop() is already an infinite loop, so avoid long while loops inside it. A long or stuck loop can freeze the game and stop button reads, servo updates, and LCD updates.

**Try it**

Print only even numbers from 2 to 10. Start with a for loop and use the remainder operator to decide which numbers should be printed.

# 6. Arrays

An array stores many values of the same type under one name. The positions, called indexes, start at 0 rather than 1.

```
int servoPins[2] = {9, 10};
int scores[2] = {0, 0};
scores[0]++;   // player 1 scores
for (int i = 0; i < 2; i++) { Serial.println(scores[i]); }
```

## Line-by-line explanation

- **int servoPins[2]** creates an integer array with space for two values. The initial values are 9 and 10, the two servo pins.
- **int scores[2]** creates two score positions and starts both at zero.
- **scores[0]++;** increases the first position. Because counting starts at zero, this is player 1's score.
- **for (int i = 0; i < 2; i++)** visits index 0 and index 1, then stops before index 2.
- **scores[i]** uses the current loop index to read each score.
- **Serial.println(scores[i]);** prints the value from each position.

**Watch out**

An array of size 2 has indexes 0 and 1 only. **scores[2]** does not exist. Reading or writing it can damage unrelated data and create confusing behaviour.

| Index | Meaning | Starting value |
|---|---|---|
| scores[0] | player 1 | 0 |
| scores[1] | player 2 | 0 |

# 7. Functions

Functions are reusable mini-programs with names. They keep loop() short and readable, and they let you solve one small problem once and use the solution many times.

```
int addPoints(int current, int points) {
  return current + points;
}
void showScore(int p1, int p2) {
  // display both scores here
}
```

## Line-by-line explanation

- **int** before addPoints is the return type. It says this function sends back an integer.
- **addPoints** is the function name. Choose a name that describes the job.
- **(int current, int points)** lists the parameters in brackets. These are input values the function receives when it is called.
- The opening brace begins the function body and the closing brace ends it.
- **return current + points;** calculates a new score and sends that value back to the place that called the function.
- **void** before showScore means that function returns nothing.
- **showScore(int p1, int p2)** receives two integer scores and can display both of them inside its body.

You could call the first function from loop() like this: **score = addPoints(score, 1);** The returned value becomes the new score. You could call the second with **showScore(scores[0], scores[1]);**.

**Try it**

Create a function that prints a welcome message. Then create another function that receives two scores and returns the larger one. Test both from loop() and print the returned result.

# 8. Serial Monitor: Your Debugging Friend

Arduino sketches do not use cout for ordinary messages. They use **Serial** to send information from the board to your computer. This gives you a window into what variables really contain while the project is running.

```
void setup() {
  Serial.begin(9600);
}
void loop() {
  Serial.print("Angle: ");
  Serial.println(pos);
  delay(200);
}
```

## Line-by-line explanation

- **void setup()** is the one-time setup function.
- **Serial.begin(9600);** starts serial communication at 9600 baud, or bits per second.
- **void loop()** repeats the reporting work.
- **Serial.print("Angle: ");** prints a label and stays on the same line.
- **Serial.println(pos);** prints the value of pos and then moves to a new line.
- **delay(200);** waits 200 milliseconds so the monitor is readable rather than flooded with messages.

Open **Tools > Serial Monitor** in the Arduino IDE and set the monitor to **9600 baud**. Use Serial to check sensor readings, button states, angles, timer values, and game states instead of guessing what the board is doing.

**Watch out**

The baud rate in the Serial Monitor must match the number in Serial.begin(9600). If they do not match, the output may look like random symbols.

# 9. Timing: delay() versus millis()

**delay(1000)** pauses the whole board for one second. During that pause, nothing else can happen: no button reads, no new sensor values, no servo decisions, and no screen updates.

**millis()** returns the number of milliseconds since the board started. By comparing the current clock value with a saved time, you can schedule work without blocking the rest of the sketch.

```
unsigned long lastUpdate = 0;
void loop() {
  if (millis() - lastUpdate >= 1000) {
    lastUpdate = millis();
    // update the timer on the LCD once per second
  }
  // servos and buttons keep working here
}
```

## Line-by-line explanation

- **unsigned long lastUpdate = 0;** creates a large, non-negative timestamp and starts it at zero.
- **if (millis() - lastUpdate >= 1000)** asks whether at least 1000 milliseconds have passed since the last update.
- **lastUpdate = millis();** remembers the current time when the update happens.
- The comment marks the place where the LCD timer can be refreshed once per second.
- The final comment reminds you that servo and button work remains available outside the timing check.

## A simple timeline

1. Check the clock.
2. When one second has passed, act.
3. Remember the new time.
4. Continue servicing everything else while the clock keeps moving.

The subtraction is used because it measures the time elapsed since lastUpdate. This pattern also remains safe when the millis counter eventually wraps around.

**Tip**

For a game like DuelMaze, prefer millis() for timers and countdowns. Use delay() only when a short, deliberate pause will not make the controls feel unresponsive.

# 10. Reading Input

Arduino inputs are commonly digital or analog. Digital inputs report one of two states. Analog inputs report a range of values.

## Digital input and output

Use **pinMode(2, INPUT_PULLUP);** to configure pin 2 as an input with Arduino's built-in pull-up resistor. Then **digitalRead(2)** returns HIGH or LOW. With a button connected between pin 2 and GND, the reading is LOW when the button is pressed. INPUT_PULLUP reverses the intuitive pressed value because the pin normally rests HIGH and the button pulls it down to ground.

digitalWrite(13, HIGH); turns an output, such as an LED, on when the output is connected appropriately. LOW turns it off.

## Analog input

**analogRead(A0)** returns a value from 0 to 1023. A potentiometer or joystick axis can produce values across that range as its position changes.

```
int angle = map(analogRead(A0), 0, 1023, 0, 180);
```

- **analogRead(A0)** gets the current knob or joystick reading.
- **0, 1023** describe the input range from the analog pin.
- **0, 180** describe the output range for a servo angle.
- **map(...)** scales the input range into the output range, and the result is stored in angle.

## Keeping values safe

**constrain(value, min, max)** keeps a value inside safe limits. For example, **angle = constrain(angle, 20, 160);** changes values below 20 to 20 and values above 160 to 160.

## Debouncing buttons

Mechanical buttons bounce briefly when pressed. The Arduino may interpret a single physical press as multiple quick presses. A common fix is to ignore a new press if it arrives within about 50 milliseconds of the last accepted press.

**Tip**

**Debounce sequence:** detect a change, remember the time, wait until about 50 ms has passed, read again, and accept the press only if the state is still pressed.

# 11. Servo Motors

A hobby servo turns to an exact angle, usually between 0 and 180 degrees. DuelMaze can use two servos to tilt the maze in two directions.

| Servo wire | Connect to |
|---|---|
| Brown or black | GND |
| Red | 5V |
| Orange or yellow | Signal pin |

```
#include <Servo.h>
Servo myServo;
int pos;
void setup() { myServo.attach(9); }
void loop() {
  for (pos = 0; pos <= 180; pos += 1) { myServo.write(pos); delay(15); }
  for (pos = 180; pos >= 0; pos -= 1) { myServo.write(pos); delay(15); }
}
```

## Line-by-line explanation

- **#include <Servo.h>** includes the Servo library so the sketch knows the servo commands.
- **Servo myServo;** creates a Servo object named myServo. Use this exact spelling everywhere.
- **int pos;** declares the integer that stores the current position.
- **myServo.attach(9);** connects myServo to signal pin 9 in setup().
- **for (pos = 0; pos <= 180; pos += 1)** starts at 0, continues through 180, and increases the angle by 1 each round.
- **myServo.write(pos);** sends the current angle to the servo.
- **delay(15);** gives the servo a short time to move before the next angle is requested.
- The second for loop starts at 180, continues while pos is at least 0, and decreases by 1 so the servo sweeps back.

## Control a servo with a knob

```
int angle = map(analogRead(A0), 0, 1023, 0, 180);
myServo.write(angle);
```

- **analogRead(A0)** reads the potentiometer or joystick axis.
- **map(...)** converts 0–1023 into the servo's 0–180 degree range.
- **myServo.write(angle)** moves the servo to the calculated angle.

**Watch out**

Two or more servos can draw more current than the Arduino 5V pin supplies, causing resets or jitter. Use an external 5V supply and connect its GND to Arduino GND. Always write **myServo**, not myservo.

**Try it**

Limit the servo to a safer range such as 20–160 degrees. Map the input into 20–160 and then use constrain to keep the final angle inside that range.

# 12. The I2C LCD (16x2)

I2C lets the LCD communicate using only two data wires, plus power and ground. On an Arduino Uno, the usual connections are:

| LCD pin | Arduino Uno |
|---|---|
| GND | GND |
| VCC | 5V |
| SDA | A4 |
| SCL | A5 |

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);  // address, columns, rows
void setup() { lcd.init(); lcd.backlight(); }
void loop() {
  lcd.setCursor(0, 0);   // column 0, row 0 (top)
  lcd.print("Hello, From");
  lcd.setCursor(0, 1);   // column 0, row 1 (bottom)
  lcd.print("Rajesh");
  delay(1000);
}
```

## Line-by-line explanation

- **#include <Wire.h>** includes the library that handles I2C communication.
- **#include <LiquidCrystal_I2C.h>** includes the commands for the I2C LCD.
- **LiquidCrystal_I2C lcd(0x27, 16, 2);** creates an LCD named lcd. 0x27 is the address, 16 is the number of columns, and 2 is the number of rows.
- **lcd.init()** starts the LCD.
- **lcd.backlight()** turns on the LCD backlight.
- **lcd.setCursor(0, 0)** places the cursor at column 0, row 0, the top-left position.
- **lcd.print("Hello, From")** writes the first message.
- **lcd.setCursor(0, 1)** moves to column 0, row 1, the bottom row.
- **lcd.print("Rajesh")** writes the second message.
- **delay(1000)** leaves the message visible for one second before loop()

repeats.

| Command | Purpose |
| --- | --- |
| lcd.clear() | Wipes the screen. |
| lcd.print(score) | Prints a number variable. |
| setCursor(column, row) | Chooses a position; both column and row count from 0. |

**Watch out**

A blank screen or only boxes often means the contrast screw on the back needs adjustment. If there is nothing at all, try address 0x3F or run an I2C scanner sketch.

**Tip**

Avoid calling lcd.clear() every loop because it causes flicker. Overwrite old text with spaces when a new message is shorter than the old one.

# 13. Putting It All Together — A Mini DuelMaze Starter

This starter sketch combines two potentiometers or joystick axes on A0 and A1, two servos on pins 9 and 10, a 16x2 I2C LCD, Serial output, and a non-blocking timer. The inputs tilt the maze, the LCD shows DuelMaze and the running time, and the Serial Monitor prints both angles.

```
#include <Servo.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

Servo servo1;
Servo servo2;

const int SERVO1_PIN = 9;
const int SERVO2_PIN = 10;

unsigned long startTime = 0;
unsigned long lastLcdUpdate = 0;

LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  servo1.attach(SERVO1_PIN);
  servo2.attach(SERVO2_PIN);
  Serial.begin(9600);
  lcd.init();
  lcd.backlight();
  startTime = millis();
  lcd.setCursor(0, 0);
  lcd.print("DuelMaze");
}

void loop() {
  int angle1 = map(analogRead(A0), 0, 1023, 0, 180);
  int angle2 = map(analogRead(A1), 0, 1023, 0, 180);

  angle1 = constrain(angle1, 0, 180);
  angle2 = constrain(angle2, 0, 180);

  servo1.write(angle1);
  servo2.write(angle2);
```

```
  Serial.print("Servo 1: ");
  Serial.print(angle1);
  Serial.print("  Servo 2: ");
  Serial.println(angle2);

  unsigned long elapsedSeconds = (millis() - startTime) / 1000;

  if (millis() - lastLcdUpdate >= 1000) {
    lastLcdUpdate = millis();
    lcd.setCursor(0, 0);
    lcd.print("DuelMaze        ");
    lcd.setCursor(0, 1);
    lcd.print("Time: ");
    lcd.print(elapsedSeconds);
    lcd.print(" s       ");
  }
}
```

## Line-by-line explanation

- **#include <Servo.h>** makes servo commands available.
- **#include <Wire.h>** makes I2C communication available.
- **#include <LiquidCrystal_I2C.h>** makes the LCD commands available.
- **Servo servo1; Servo servo2;** creates two servo objects, one for each tilt direction.
- **const int SERVO1_PIN = 9;** and **const int SERVO2_PIN = 10;** give readable names to the signal pins and prevent accidental changes.
- **unsigned long startTime** remembers when the game began, while **lastLcdUpdate** remembers the most recent screen update.
- **LiquidCrystal_I2C lcd(0x27, 16, 2)** creates a 16-column, 2-row LCD at address 0x27.
- **servo1.attach** and **servo2.attach** connect the two objects to their pins.
- **Serial.begin(9600)** starts the Serial Monitor connection at 9600 baud.
- **lcd.init()** starts the LCD, and **lcd.backlight()** turns on its light.
- **startTime = millis()** records the starting clock value.
- The first cursor and print commands show DuelMaze on the top row when setup finishes.
- **analogRead(A0)** and **analogRead(A1)** read the two controls.
- **map(...)** converts each input range of 0–1023 into a servo range of 0–180.
- **constrain** keeps both calculated angles inside the safe 0–180 range.
- **servo1.write(angle1)** and **servo2.write(angle2)** send the angles to the two servos.

- The Serial commands print both angle labels and values so you can inspect the controls while testing.
- **elapsedSeconds = (millis() - startTime) / 1000** measures elapsed milliseconds and converts them to whole seconds.
- **if (millis() - lastLcdUpdate >= 1000)** waits until one second has passed without stopping the controls.
- **lastLcdUpdate = millis()** remembers the time of the current LCD update.
- **lcd.print("DuelMaze ")** rewrites the first row and uses spaces to remove leftover characters.
- The second row prints Time:, the elapsed seconds, and extra spaces so an older longer value is overwritten.
- The closing brace ends the timing check, and the final brace ends loop(). The board then starts loop() again immediately.

## Program flow

1. Read the two analog inputs.
2. Map the readings to servo angles.
3. Constrain both angles.
4. Write the angles to the servos.
5. Print both angles to Serial.
6. Calculate elapsed time.
7. Update the LCD timer every second.
8. Keep looping so the controls remain responsive.

**Try it**

Make one safe modification at a time: reverse one axis by mapping 1023 to 0 and 0 to 180, change the LCD message, or change the update interval from 1000 milliseconds to another value.

# 14. Common Errors and How to Fix Them

Errors are normal feedback from the tools. Start with the first message, make one change, and verify again.

| Error or symptom | Likely cause | Fix |
|---|---|---|
| “expected ';' before...” | Missing semicolon on the line above. | Check the previous line and add the semicolon where the statement ends. |
| “was not declared in this scope” | Spelling or case mismatch, or a variable declared inside another block. | Check spelling and capitalisation, and move the declaration to a scope where it is available. |
| “No such file or directory” for a library | The required library is not installed. | Install it from Library Manager. |
| Upload fails | Wrong board or COM port selected in Tools. | Choose the correct board and COM port in Tools, then try Upload again. |
| Servo jitters or board resets | Power supply issue. | Check the power arrangement, use an external 5V supply for multiple servos, and connect grounds together. |
| LCD shows nothing | Address, wiring, or contrast. | Check the wiring, adjust contrast, try 0x3F, or run an I2C scanner sketch. |

# Calm debugging checklist

- [x] ~~Read the first error, not only the last one.~~
- [x] ~~Check the line above the highlighted line.~~
- [x] ~~Check spelling and capitalisation, including myServo and myservo.~~
- [x] ~~Simplify the sketch temporarily.~~
- [x] ~~Print values in Serial Monitor.~~
- [x] ~~Test one part at a time.~~
- [x] ~~Reconnect the pieces after each part works.~~

**Remember**

A useful debugging question is: “What did I expect, and what actually happened?” The difference tells you what to measure next.

# 15. Quick Reference Cheat Sheet

Use this page as a compact reminder after you have worked through the explanations. It is a reference, not a replacement for the examples and debugging practice earlier in the guide.

## Data types

| Type | Use |
|---|---|
| int | Whole numbers |
| long / unsigned long | Large counts and millis() times |
| float / double | Decimals |
| bool | true or false |
| char | One character |
| String | Text |
| byte | 0 to 255 |

## Operators

| Group | Reference |
|---|---|
| Arithmetic | + add, - subtract, * multiply, / divide, % remainder |
| Comparison | == equal, != not equal, < less, > greater, <=, >= |
| Logic | && both, \|\| at least one, ! reverse |
| Shortcuts | x += 5, x -= 1, x++, x-- |

## Control structures

```
if (condition) { ... } else { ... }

for (int i = 0; i < 5; i++) { ... }

while (condition) { ... }

int values[2] = {0, 0};
values[0]

int add(int a, int b) { return a + b; }
```

## Serial and timing

| Pattern | Purpose |
| --- | --- |
| Serial.begin(9600); | Start communication. |
| Serial.print(value); | Print and stay on the line. |
| Serial.println(value); | Print and move to a new line. |
| if (millis() - last >= interval) { last = millis(); } | Run a task without blocking. |

## Servo and LCD commands

| Device | Useful commands |
| --- | --- |
| Servo | myServo.attach(9); myServo.write(angle); |
| LCD | lcd.init(); lcd.backlight(); lcd.clear(); |
| LCD position | lcd.setCursor(column, row); |
| LCD output | lcd.print(message); lcd.print(score); |
| Analog input | analogRead(A0), map(...), constrain(...) |
| Digital input | pinMode(2, INPUT_PULLUP); digitalRead(2) |

**Fast reminder**

Verify first, upload second, observe the result, and change one thing at a time. Small tests make large projects manageable.

# Practice Challenges

Use one small experiment at a time. Write down what you expect, test it, and use Serial Monitor or the LCD to observe the result.

1. **Make the LCD count down from 10.** Goal: show 10, 9, 8, and so on until 0. Hint: use a number variable, a loop or millis(), and lcd.setCursor().
2. **Make a servo move only while a button is held.** Goal: read a button with INPUT_PULLUP and move the servo only when the reading is LOW. Hint: place the servo command inside an if condition.
3. **Show the higher of two scores on the LCD.** Goal: compare two score variables and print the larger one. Hint: use if/else or a function that returns the larger value.
4. **Build a 30 second round timer that prints GAME OVER.** Goal: let the timer run without blocking the controls. Hint: compare millis() with a saved start time and use if to detect when 30 seconds have passed.

## Build, break, debug, repeat.

Errors are information, not proof that you cannot code. Every working project begins as a collection of small tests, and every test teaches you what to try next.

---